CSC 251 Lecture 04: Functions, Global and Local Variables

Python built-in functions found at: https://docs.python.org/3/library/functions.html

Math module functions and constants at: https://docs.python.org/3/library/math.html

```
import math
. . .
print(2 * math.pi * radius)
```

module

constant or method

```
from math import pi
. . .
pi
```

A function **must** possess:

* Name (usually)
* Purpose

might possess:

* parameters
* return

Advantages:

- Usability
- Efficiency
- Reability

Function names:

- Should describe its task
- No reserved words or spaces
- First character must be letter or underscore
- All other characters must be a letter, number, or underscore
- Case-sensitive
- `camel_case()` is the Python convention

Function definitions and flow of execution:

- Function definitions must appear before it is called but they can be defined in any order
- Execution always begins at the first statement NOT included in the functions
- Functions are executed only when called
- A function call transfers control to the first statement in the body of the called function
- The function statements are each executed then control is passed back to the code immediately following the function call

Void functions

```
def function_name():
    statement
    statement

function_name()
```

indented w/in function

Example 1:

```
def send_hurricane_alert():
    print("Hurricane warning, expect heavy rain and strong wind!")

send_hurricane_alert()
```

Example 2:

```
def main():
    print("Wolfpack Alert:")
    send_hurricane_alert()
    print("Please exercise caution!")

def send_hurricane_alert():
    print("Hurricane warning, expect heavy rain and strong wind!")

if __name__ == '__main__':
    main()
```

double underscore

execution starts here

What's that last line all about? Why should you include this?

allows you to ignore main and use as a module.

Your file containing functions may be used as a module. Modules can be imported into other programs or run as stand-alone code. If imported into another program, you don't want to run the main program of the module…

To prevent the main function from being executed if the file is imported as a module, you can conditionally execute main. Each module executes its main function only when the module is being run as the main program:

- When a source code file is loaded into the Python interpreter, a special variable called `__name__` is created (those are each 2 underscores, BTW)
- If the source code file has been imported as a module, the `__name__` variable will be set to the name of the module.
- If the source code file is being executed as the main program, the `__name__` variable will be set to the value `'__main__'`.

Passing arguments to a function

Example 3:

```
def main():
    first = input("Enter first name: ")
    last = input("Enter last name: ")
    print_name(first, last)

def print_name(fname, lname):
    print("Hello", fname, lname + "!")
. . .
```

arguments

parameters

Positional arguments versus keyword arguments

Example 4:

```
def main():
    multiply_then_add(1,2,3)
    multiply_then_add(y=2,z=3,x=1)
    multiply_then_add(1,z=3,y=2)

def multiply_then_add(x,y,z):
    print(x*y+z)

if __name__ == '__main__':
    main()
```

positional

Keyword

combo with positional first

How does Python pass parameters? Pass-by- object ? reference

simple / immutable vars ⇒ pass-by-value

complex / mutable objects ⇒ pass-by reference

Example 5: How does this work? Looks like pass-by- value

```
def main():
    value = 99
    change_me(value)
    print(f"value = {value} in main")

def change_me(value):
    value = 0
    print(f"value = {value} in change_me")

if __name__ == '__main__':
    main()
```

value (local) 99

value (local) 0

Output

value = 0 in change_me

value = 99 in main

Value-returning functions

Example 6:

```
def main():
    value = 99
    value = change_me(value)
    print(f"value = {value} in main")

def change_me(value):
    value = 0
    print(f"value = {value} in change_me")
    return value

if __name__ == '__main__':
    main()
```

value 99 0

value 0

Global versus Local Variables

The good, the bad and the ugly of global variables:

- Defined before function definitions
- Read-only in functions unless preceded by global
- Conclusion? Use with extreme caution!
- Beware of shadowing
- Please use!
- Global constants (use ALL CAPS)

Example 7a:

```
age = 19

def main():
    vote_determiner()
    print("Age:", age)

def vote_determiner():
    if age >= 18:
        print("You are eligible to vote!")
    else:
        print("You are not eligible to vote, sorry!")

if __name__ == '__main__':
    main()
```

global

scope

use here

and here

Output:

```
You are eligible...
Age: 19
```

Example 7b:

```
age = 19

def main():
    vote_determiner()
    print("Age:", age)

def vote_determiner():
    age = int(input("Enter your current age: "))
    if age >= 18:
        print("You are eligible to vote!")
    else:
        print("You are not eligible to vote, sorry!")
```

scope

local var shadows global

Output:

```
Enter...: 14
You are not eligible...
Age: 19
```

Example 7c:

```
age = 19

def main():
    vote_determiner()
    print("Age:", age)

def vote_determiner():
    global age
    age = int(input("Enter your current age: "))
    if age >= 18:
        print("You are eligible to vote!")
    else:
        print("You are not eligible to vote, sorry!")
. . .
```

scope

global

not local

Output:

Enter . . . : 14
You are not . . .
Age: 14

Example 7d:

no global vars

```
def main():
    age = int(input("Enter your current age: "))
    vote_determiner(age)

def vote_determiner(voter_age):
    if voter_age >= 18:
        print("You are eligible to vote!")
    else:
        print("You are not eligible to vote, sorry!")
. . .
```

passed

local

Output:

Enter . . . : 14
You are not . . .

What if `voter_age` is changed in the function? Does it change `age`?

You must return it!

Example 8:

```
def doIt(x):
    if x > 5:
      return 2*x

print(doIt(6))
print(doIt(4))
```

Output:

12
None

← Python's way to prevent a crash

Improvements?

include catch-all return !

Example 9: How can we complete the following?

```
def main():
    fname, lname = get_name()
    quiz = 9
    print(f"Quiz grade for {fname} {lname}: {quiz}/10")

def get_name():
    first = input("Enter first name: ")
    last = input("Enter last name: ")
    <processing completed on names>
    return first, last

if __name__ == '__main__':
    main()
```

unpack the tuple

packing tuple

parentheses optional

The Python Type System

While Python is dynamically typed, recent versions (3.5+) now support type hints or type annotations.

- The interpreter checks the types of values at runtime before performing operations since values must have a type and obey typing rules.
- Type annotations assist in using libraries, classes, and functions
- Enable use of static type checkers in IDEs, mypy, or with FastAPI and other packages
- Simple types include: int, float, bool, str, bytes
- Generic types include: dict, list, set, tuple
- Older Python versions (prior to 3.9) required:

```
from typing import List
```

Static typing documentation: https://typing.readthedocs.io/en/latest/index.html
Reasons to avoid static typing: https://typing.python.org/en/latest/guides/typing_anti_pitch.html
A nice reference: https://mypy.readthedocs.io/en/stable/cheat_sheet_py3.html#variables

Example 10:

```
def is_even(number):
    return number % 2 == 0
```

versus

var type

return type

```
def is_even(number: int) -> bool:
    return number % 2 == 0
```

Testing Functions

Example 11: For non-returning functions, visually compare the expected and actual output.

```
def multiply_then_add(x,y,z):
    print(x*y+z)

def test_multiply_then_add():
    print("Testing multiply_then_add(1,2,3)")
    expected = 5
    print("Expected: ", expected)
    print("Actual: ", end ='')
    multiply_then_add(1,2,3)

test_multiply_then_add()
```

Compare

Example 12: For value-returning functions, check the expected and actual output.

```
def is_even(number: int) -> bool:
    return number % 2 == 0

def test_is_even():
    print("Testing is_even(27)")
    expected = False
    actual = is_even(27)
    if expected == actual:
        print("Test passed")
    else:
        print("Test failed")
        print("Expected: ", expected)
        print("Actual: ", actual)

test_is_even()
```

More automated if returned

Lambda Functions

Small, concise, anonymous functions containing a single expression, no statements.

Example 13a: Could be named, although not best practice

```
title = lambda col1, col2: f'{col1:^18}{col2:^18}'
print(title('Home', 'Away'))
```

Example 13b:

```
format_num = lambda num: f'{num:e}' if isinstance(num, int) else f'{num:,.2f}'
print(format_num(1000000)
print(format_num(23094830249830.123)
```

Example 13c: Best when used with functions like `map()`, `filter()`, and `reduce()`

```
. . .
sq_double = list(map(lambda x: (2 * x) ** 2, list_name)
```

IIFE ("iffy") execution – Immediately Invoked Function Expression

```
(lambda x: (2 * x) ** 2) (3)
```

In an interactive Python interpreter, we can define the lambda function without giving it a name:

```
lambda x: (2 * x) ** 2
```

Then soon after:

```
_(3)
```

The interpreter binds the underscore (_) to the last lambda function.

To use or not to use lambda functions:

Advantages

- Convenient
- Support positional & keyword args
- Quick
- Reusable
- * Reduces clutter

Disadvantages

- Difficult to modularize
- Expressions, not statements
- No difference in performance
- Traceback only id's as "lambda"
- Poor readability
- * Unfamiliar